

ACKNOWLEDGMENT

Dreams never turn into reality unless a lot of efforts and hard work is put into it. It takes a lot of efforts to make your way to the goal and having someone to guide you and help you is always a blessing. So, we acknowledge all those whose guidance and encouragement served as a beacon light and crowned our efforts with success.

We are thankful to **Dr. Srikantappa A S, Principal**, CIT, Mandya, for all the support he has rendered for providing us an opportunity to carry out our Major Project.

We would like to thank **Prof. Arpitha S K, Assistant Professor and Head of Department**, Department of Computer Science and Engineering, for her support for the successful completion of our major project.

We would like to sincerely thank our project co-ordinator, **Prof. Sahana G C, Assistant Professor**, Department of Computer Science and Engineering, CIT, Mandya, for her constant incitement and advice throughout the project.

We would like to sincerely thank our internal guide **Prof. Chaithra N T, Assistant Professor**, Department of Computer Science and Engineering for providing relevant information, valuable guidance and encouragement to complete the Major Project.

Abhishek Kumar T [4CA23CS002]
Javeriya Khan [4CA23CS048]
Keerthana R [4CA23CS059]

ABSTRACT

Electronic mail remains the dominant channel for official and personal communication, yet it is also the primary vector for phishing, malware, promotional flooding and bulk spam. Conventional binary filters separate mail into spam versus ham and give users little insight into why a message was flagged. This project presents **AI-Powered Email Spam Detection and Threat Analysis**, a Flask-based system that connects to a user's Gmail mailbox through Google OAuth 2.0 (gmail.readonly scope) and classifies every message into six categories -- Spam, Not Spam, Promotion, Newsletter, Phishing and Malware -- using a hybrid of a fine-tuned RoBERTa-base transformer and a five-model classical ensemble (Multinomial Naive Bayes, Logistic Regression, Random Forest, Gradient Boosting and MLP) fused with 0.90/0.10 weighted soft voting. Each prediction is accompanied by a 0-100 risk score, a Low/Medium/High level, an urgency estimate from curated regex patterns, and a human-readable breakdown of the factors that drove the decision. Two adaptive loops personalise the filter without retraining: sender reputation tracking aggregates per-sender history for 95 senders, and learned-keyword memory retains 232 risky terms, both stored in SQLite via SQLAlchemy and evolved through five Alembic migrations. The pipeline -- OAuth consent, batched Gmail retrieval with retry-on-429, MIME parsing and bleach sanitisation, dual-model inference, risk assembly and persistence -- runs on a cancellable background thread and completes a fifty-message scan in 30-60 seconds on CPU. A Bootstrap 5.3 interface provides a dashboard, filterable results, Chart.js analytics and a threat console that ranks the most dangerous messages and repeat offenders. Evaluation on a shared stratified 3,003-message holdout reports approximately 98% weighted accuracy, precision, recall and F1 for the fused system, with RoBERTa contributing contextual phishing sensitivity and the ensemble contributing lexical stability. Security hardening includes security headers on every response, bleach whitelisting, SQLAlchemy parameter binding and per-endpoint rate limiting, with a documented backlog for secret management and CSP tightening.

CONTENTS

Acknowledgment	I
Abstract	II
Table Of Contents	Page no
Chapter 1 : Introduction	1-3
1.1 Problem Statement	1-2
1.2 Objectives	1-2
1.3 Project Scope	2-3
1.4 Limitations	2
1.5 Proposed Solution Overview	2-3
1.6 Report Organisation	3
Chapter 2 : Literature Survey	4-5
2.1 Introduction	4
2.2 Related Work	4
2.3 Research Gap and Positioning	5
Chapter 3 : System Overview and Requirements	6-9
3.1 Technology Stack	6
3.2 Module Architecture	6-7
3.3 Data Flow Overview	7
3.4 Functional Requirements	7
3.5 Non-Functional Requirements	7
3.6 Hardware and Software Requirements	8
3.7 Deployment Architecture	8
3.8 Preprocessing Engine	8-9
Chapter 4 : System Design	10-14
4.1 System Architecture	10
4.2 Modules and Explanation	10-11
4.3 Web Routes and API Endpoints	11
4.4 Database Design	12
4.5 User Interface (UI)	13

4.6 Working Procedure	13
4.7 Authentication and Authorisation	14
4.8 Background Task Engine	14
Chapter 5 : Implementation	15-24
5.1 RoBERTa Transformer Classifier	15
5.2 Ensemble Classification	15
5.3 Fusion, Risk and Urgency Scoring	16
5.4 Base Risks and Historical Baselines	16-17
5.5 Adaptive Intelligence	17-18
5.5.1 Adaptive Intelligence Configuration	18
5.6 Data Collection and Description	18
5.6.1 Dataset Audit Script	19
5.6.2 Engineered Features and Preprocessing Steps	19
5.6.3 Dataset Size Justification	20
5.7 Data Processing	20
5.7.1 Overall Processing	20
5.7.2 Textual Processing	20
5.7.3 Feature Formation	20-21
5.8 Proposed Model	20-21
5.8.1 Model Singleton Management	22
5.8.2 Ensemble Class Probabilities	23
5.8.3 Risk Score Composition	23
5.8.4 Fallback Strategy	23-24
Chapter 6 : Testing and Security Analysis	25-29
6.1 Introduction to Testing	25
6.2 Types of Testing	25-27
6.2.1 Unit Testing	25-26
6.2.2 Integration Testing	26
6.2.3 System Testing	26-27
6.3 Security Audit (OWASP)	28
6.4 Edge Cases and Failure Handling	28
6.5 Performance Considerations	29

Chapter 7 : Results	30-34
7.1 Evaluation Methodology and Model Results	30
7.2 Screenshots	30-34
CONCLUSION	VI
FUTURE SCOPE	VII
REFERENCES	VIII

LIST OF FIGURES

FIGURE NO	DESCRIPTION	Page no
4.1	System Architecture	10
4.2	ER Diagram	12
5.1	Dataset Distribution	19
5.2	Dual-Model Pipeline	21
5.3	Data Flow	21
5.4	Workflow	22
7.1	Landing Page	31
7.2	Login Page	31
7.3	Dashboard	32
7.4	Results Summary	32
7.5	Results Table	33
7.6	Email Detail View	33
7.7	Analytics	34
7.8	Threat Console	34

CHAPTER 1

Introduction

Electronic mail remains the most widely used channel of digital communication, and it is also the primary delivery vehicle for unsolicited bulk mail, phishing campaigns, and malware. Conventional mailbox filters separate mail into a small number of fixed folders and offer the user little insight into why a message was treated as dangerous. This project, developed for the academic year 2026-2027, presents an AI-Powered Email Spam Detection and Threat Analysis system that connects to a user's Gmail account through Google OAuth 2.0, retrieves recent messages through the Gmail API v1, and classifies every message into one of six categories - Spam, Not Spam, Promotion, Malware, Newsletter, and Phishing - using a hybrid of a fine-tuned RoBERTa-base transformer and a five-model classical ensemble. Every prediction is accompanied by a 0-100 risk score, a risk level, an urgency estimate, and a human-readable explanation of the factors that drove the decision.

The system is delivered as a Flask 2.3.3 web application backed by SQLAlchemy and SQLite, with a responsive Bootstrap 5.3 interface that provides a dashboard, sortable results, Chart.js analytics, and a threat console. Adaptive intelligence mechanisms - sender reputation tracking and learned-keyword memory stored in the database - allow the filter to personalise itself to an individual inbox without retraining the underlying models. The complete pipeline from OAuth consent to explained prediction runs automatically on a background thread, so a fifty-message inbox scan completes in roughly thirty to sixty seconds on CPU.

1.1 Problem Statement

Manually triaging a busy inbox is slow and error-prone, and a binary spam-versus-ham decision is no longer sufficient. Modern inboxes face multi-dimensional threats: phishing messages that impersonate trusted brands to steal credentials, malware carriers that exploit curiosity to deliver payloads, promotional floods that waste attention, and newsletters that hide tracking. Users need an automated system that understands message semantics, assigns a fine-grained category, quantifies the danger on a continuous scale, and explains its reasoning, without requiring hand-written filter rules or exposing the user's credentials to the application.

1.2 Objectives

The specific objectives realised by this project are: (i) to fine-tune a RoBERTa-base transformer for six-way email classification on a 30,028-message corpus; (ii) to train a

complementary five-model classical ensemble on TF-IDF and engineered features; (iii) to fuse both models with a 0.90/0.10 weighted soft-voting strategy with graceful degradation when either model is unavailable; (iv) to attach an explainable risk score that combines model confidence, category severity, urgency detection, sender reputation, and learned keywords; (v) to integrate securely with Gmail through OAuth 2.0 using the read-only scope; and (vi) to present results through a dashboard, analytics charts, and a threat console.

1.3 Project Scope

The scope covers the full pipeline: OAuth authentication, Gmail retrieval in rate-limit-safe batches, MIME parsing and HTML sanitisation, dual-model inference, multi-factor risk assembly, persistence in four database tables, and presentation across eleven Jinja2 templates. The study focuses on English-language text content. Attachment binaries, embedded images, and non-Latin scripts are outside the current scope, as is real-time push synchronisation, since analysis is user-initiated.

1.4 Limitations

Several limitations are acknowledged. RoBERTa inference on CPU costs roughly 200-500 ms per message, so a fifty-message batch spends ten to twenty-five seconds in the transformer alone. The in-memory rate limiter resets on server restart and is not shared across workers. The SECRET_KEY falls back to a development value when its environment variable is missing, OAuth tokens rest in the Flask session without encryption at rest, and the threat console route performs no administrative role check. The classifier is text-only, so image-based spam and malicious attachments are invisible to it. Finally, the default SQLite database suits single-user evaluation rather than concurrent production use, although the SQLAlchemy layer keeps a migration to PostgreSQL open.

1.5 Proposed Solution Overview

The proposed solution pairs two independent classification engines with a behavioural enrichment layer. Raw message text is scored by a fine-tuned RoBERTa-base transformer, which captures contextual deception cues such as impersonation phrasing, and simultaneously by a soft-voting ensemble of five classical machine learning models built on TF-IDF and engineered surface features, which is highly reliable on lexically distinctive content such as promotions and newsletters. A fixed 0.90/0.10 weighted fusion produces the six-way category and its confidence. The decision is then enriched by three adaptive factors computed from database history and curated pattern matchers: an urgency score from time-pressure regex

patterns, a sender reputation score from per-sender historical aggregates, and a learned-keyword boost from terms previously observed in risky mail. The result delivered to the user is therefore not a bare label but a fully explained threat assessment, accessible through an authenticated web dashboard, a Chart.js analytics view, and a threat console.

1.6 Report Organisation

The remainder of this report is organised as follows. Chapter 2 surveys the IEEE literature that grounds and differentiates the project. Chapter 3 states the system requirements and the technology stack. Chapter 4 details the design and implementation, covering architecture, routing, both classification engines, risk scoring, adaptive intelligence, data engineering, the database schema, and the user interface. Chapter 5 presents evaluation methodology and live deployment results. Chapter 6 reports the security analysis and failure handling. Chapter 7 concludes, and Chapter 8 lays out the future-roadmap in short-term, model-level, security, and infrastructure tracks.

CHAPTER 2

Literature Survey

2.1 Introduction

Spam filtering research has progressed from keyword rules and DNS blocklists through probabilistic classifiers to deep transformer models. The IEEE papers collected in this project's research folder, catalogued in `IEEE_Papers_Analysis.md`, trace this evolution between 2023 and 2025 and provide the baseline against which the present system is positioned. This chapter summarises the six primary papers studied and identifies the gap that this project fills.

2.2 Related Work

Classifying Email as High and Low Risk: An Effective Approach (ICCUBEA 2023) introduced risk-tiered spam filtering using an ensemble of Multinomial Naive Bayes, SVM, and Decision Tree, reaching 98.95 percent accuracy on binary classification with a high/low risk split. The present project generalises that idea from two risk tiers to a continuous 0-100 score with a per-email factor breakdown, and extends three ensemble members to five plus a transformer.

Email Spam Classification using DistilBERT (HNICEM 2023) applied a compressed BERT variant to email spam and reported 95.99 percent accuracy with a Matthews correlation coefficient of 92.02 percent, demonstrating that transformer language models transfer well to mail content. This project adopts the larger RoBERTa-base backbone and pairs it with a classical ensemble rather than relying on the transformer alone. A machine learning benchmark study (ICCPCT 2025) compared Naive Bayes, SVM, Decision Tree, Random Forest, and deep models on TF-IDF features individually; the present system instead combines five such estimators in a soft-voting ensemble so that their errors cancel.

An XGBoost-centred study (ICACT 2025) achieved 97 percent accuracy with an AUC of 0.97 using gradient boosting on email features, and Ensemble Learning-Based Approach for Email Spam Detection (MENACOMM 2025) reported 98.1 percent with soft voting and 98.2 percent with stacking over SVC, MLP, and Extra Tree classifiers, confirming that ensembles outperform single classifiers. Intelligent Email Spam Detection (ICTMIM 2025) compared five classical models and found SVM best at 98.45 percent. All six papers stop at binary classification; none implements urgency detection, sender reputation, adaptive keyword memory, continuous risk scoring, or live mailbox integration.

2.3 Research Gap and Positioning

The surveyed literature establishes strong binary detectors but leaves four gaps that this project addresses directly: fine-grained six-class labelling that separates phishing and malware from ordinary spam; explainable continuous risk scoring rather than a binary verdict; behavioural signals such as urgency phrasing, sender history, and learned keywords layered on top of content classification; and end-to-end integration with a real mailbox through OAuth rather than evaluation on static datasets alone. The project's comparison analysis records approximately 98 percent weighted accuracy, precision, recall, and F1 for the fused system, which is competitive with the best published binary results while offering strictly richer output.

Paper (venue, year)	Models	Best result	Classes
High/Low Risk Classification, ICCUBEA 2023	Naive Bayes, SVM, Decision Tree ensemble	98.95% accuracy	Binary + 2 risk tiers
DistilBERT Spam Classification, HNICEM 2023	DistilBERT transformer	95.99% accuracy, MCC 92.02%	Binary
ML-based Email Spam Detection, ICCPCT 2025	NB, SVM, DT, RF, deep learning compared	per-model benchmark	Binary
XGBoost Email Spam Study, ICACT 2025	Decision Tree, KNN, XGBoost	97% accuracy, AUC 0.97	Binary
Ensemble-Based Spam Detection, MENACOMM 2025	SVC + MLP + Extra Trees	98.1% voting / 98.2% stacking	Binary
Intelligent Email Spam Detection, ICTMIM 2025	RF, LR, SVM, DT, NB compared	98.45% (SVM best)	Binary
This project	RoBERTa + 5-model soft-voting ensemble	~98% weighted, with 0-100 risk scoring	Six classes + urgency + reputation

Table 2.1: IEEE Comparison

CHAPTER 3

System Overview and Requirements

The system is a monolithic but modular Flask application organised into an application layer (app.py, approximately 1,419 lines), a machine-learning package (models/), a service and utility package (utils/), a persistence layer (SQLAlchemy models plus Alembic migrations), and a presentation layer (templates/ and static/). The technology stack is summarised below.

3.1 Technology Stack

Layer	Technology	Role
Web framework	Flask 2.3.3, Werkzeug 2.3.7	HTTP routes, sessions, error pages
ORM / migrations	Flask-SQLAlchemy 3.0.5, Flask-Migrate 4.0.5	Schema management, versioned Alembic migrations
Auth / mail	Authlib 1.2.1, google-api-python-client 2.100.0	Google OAuth 2.0 and Gmail API v1
Deep learning	PyTorch 2.0-2.2, Transformers 4.30-4.36	RoBERTa-base fine-tuned classifier
Classical ML	scikit-learn 1.2-1.6, XGBoost 1.7.6, joblib	TF-IDF ensemble training and serving
Text processing	NLTK 3.8.1, BeautifulSoup4, bleach, TextBlob	Cleaning, MIME parsing, sanitisation
Data	pandas 2.0.3, numpy 1.24.3, SQLite emails.db	Dataset handling and persistence
Frontend	Bootstrap 5.3, Font Awesome, Chart.js	Dashboard, analytics, threat console
Deployment	Gunicorn 21.2.0, Procfile, Python 3.11.9	Production serving (deploy.sh for Heroku)

Table 3.1: Tech Stack

3.2 Module Architecture

app.py hosts every HTTP route, the per-IP rate limiter, the background analysis engine, sender-reputation updates, and timezone localisation helpers (IST by default, overridable per user via a cookie). The models package contains the SQLAlchemy definitions (email_model.py), the RoBERTa wrapper (roberta_model.py) with a module-level singleton, the ensemble training script (ensemble_train.py), the transformer training script (roberta_train.py), the shared evaluation harness (evaluate.py), and the fusion predictor (predictor.py). models/utils/preprocessing.py is the single source of truth for text cleaning and engineered features. The utils package provides auth.py (Authlib OAuth registration), gmail_client.py (a 505-line Gmail API wrapper), helpers.py (724 lines covering sanitisation,

urgency scoring, keyword learning, and risk breakdown assembly), and `ai_explanation.py`. The `migrations` package records five schema revisions.

3.3 Data Flow Overview

End to end, data moves as follows: the user authenticates via Google OAuth 2.0 with the `openid`, `email`, `profile`, and `gmail.readonly` scopes, and the resulting refresh token is stored in the `oauth_store` table; starting an analysis spawns a background thread that fetches the fifty most recent messages in batches of twenty with `retry-on-429` backoff; each message is parsed from MIME into headers, body, and snippet; the cleaned text is scored by RoBERTa and the ensemble; the fused probabilities determine the category and confidence; urgency regexes, sender reputation, and learned-keyword boosts are added to produce the final risk score and breakdown; the row is persisted, sender statistics are updated, and the interface renders results, analytics, and the threat console from the database.

3.4 Functional Requirements

ID	Functional requirement	Implementation
FR-1	Authenticate users with Google OAuth 2.0	<code>auth.py</code> + <code>/auth/google</code> + <code>/callback/google</code>
FR-2	Fetch the user's recent Gmail messages	<code>gmail_client.py</code> <code>get_recent_emails</code> (batches of 20)
FR-3	Classify each message into six categories	<code>predictor.py</code> fused RoBERTa + ensemble
FR-4	Compute a 0-100 risk score with level and urgency	<code>predictor.py</code> <code>_calc_risk</code> + <code>helpers.calculate_urgency</code>
FR-5	Persist results and adaptive state	<code>email_model.py</code> tables via SQLAlchemy
FR-6	Provide dashboard, results, analytics, threat console	Flask routes + Jinja2 templates + Chart.js
FR-7	Allow manual text analysis without mail access	<code>/api/analyze_text</code> endpoint
FR-8	Support bulk analysis of pasted message sets	<code>/bulk_analyze</code> endpoint

Table 3.2: Functional Requirements

3.5 Non-Functional Requirements

The non-functional requirements realised in the codebase are: performance - a fifty-message batch completing in under about one minute on CPU, achieved through batched transformer inference and model singletons; security - OAuth-only authentication, security headers on every response, whitelist-based HTML sanitisation with bleach, and per-endpoint rate limiting; reliability - `retry-on-429` backoff for Gmail, per-message error skipping, and dual-model fallback; maintainability - a single shared preprocessing module, versioned Alembic migrations, and a Procfile/Gunicorn deployment recipe; and usability - a responsive Bootstrap 5.3 interface with colour-coded risk levels and timezone-correct localisation.

3.6 Hardware and Software Requirements

Component	Requirement	Rationale
Python	3.11.9 (pinned in runtime.txt)	Reproducible runtime
Flask	2.3.3	Lightweight web framework
PyTorch	2.6.0+cpu	RoBERTa inference; CPU-only deployment
Transformers	4.36.2	RoBERTa tokenizer and model loading
SQLAlchemy	2.0.36	ORM for email_model.py tables
Alembic	1.14.1	Schema version control
Gunicorn	23.0.0	Production WSGI server
Authlib	1.3.2	Google OAuth 2.0 integration
bcrypt	4.2.1	Password hashing (admin panel)
bleach	6.2.0	HTML sanitisation whitelist
Chart.js	4.4.7 (CDN)	Analytics visualisation
Heroku-22	Heroku stack	Deployment target via Procfile

Table 3.3: HW/SW Requirements

3.7 Deployment Architecture

The application is deployed as a single Gunicorn process on Heroku-22, started via gunicorn app:app with a binding to 0.0.0.0:\$PORT and a three-hundred-second timeout. The Procfile, runtime.txt (Python 3.11.9), and deploy.sh are the three deployment artefacts. deploy.sh performs a system-level update, installs Python 3.11, creates and activates a virtual environment, installs all dependencies from requirements.txt, runs the train-and-evaluate pipeline as a pre-boot step, and starts Gunicorn. The model artefacts (best_roberta_model.pth at 476 MB, Best_Ensemble_Model.pkl at 192 MB) are committed to the repository and loaded once at startup, cached in memory, and reused for every request.

3.8 Preprocessing Engine

The preprocessing engine in models/utils/preprocessing.py provides the shared preprocess_text function used by training, evaluation, and serving code. The pipeline is: lower-case the raw text; mask URLs, email addresses, and phone numbers with placeholder tokens (URL, EMAIL, PHONE); collapse repeated punctuation (!!!! becomes !) and repeated whitespace; remove non-alphabetic characters; and return the cleaned string. TF-IDF vectorisation is performed by a pre-fitted TfidfVectorizer loaded from models/tfidf_vectorizer.pkl; the vectoriser is never refitted at inference time. The six engineered features (length ratio, special-character ratio, capital ratio, URL flag, phone flag, urgency score) are appended to the sparse TF-IDF row before the ensemble sees it. RoBERTa

receives only the cleaned text through its own tokenizer and never sees the engineered features.

CHAPTER 4

System Design

4.1 System Architecture

The architecture separates presentation, application, service, ML engine, and data layers, with two external services (Google OAuth and the Gmail API) at the boundary. Each layer communicates only with its neighbours: routes call utilities, utilities call the predictor, and the predictor reads the model artefacts, while routes and utilities read the database through the ORM. This layering keeps the transformer, the ensemble, and the storage engine independently replaceable.

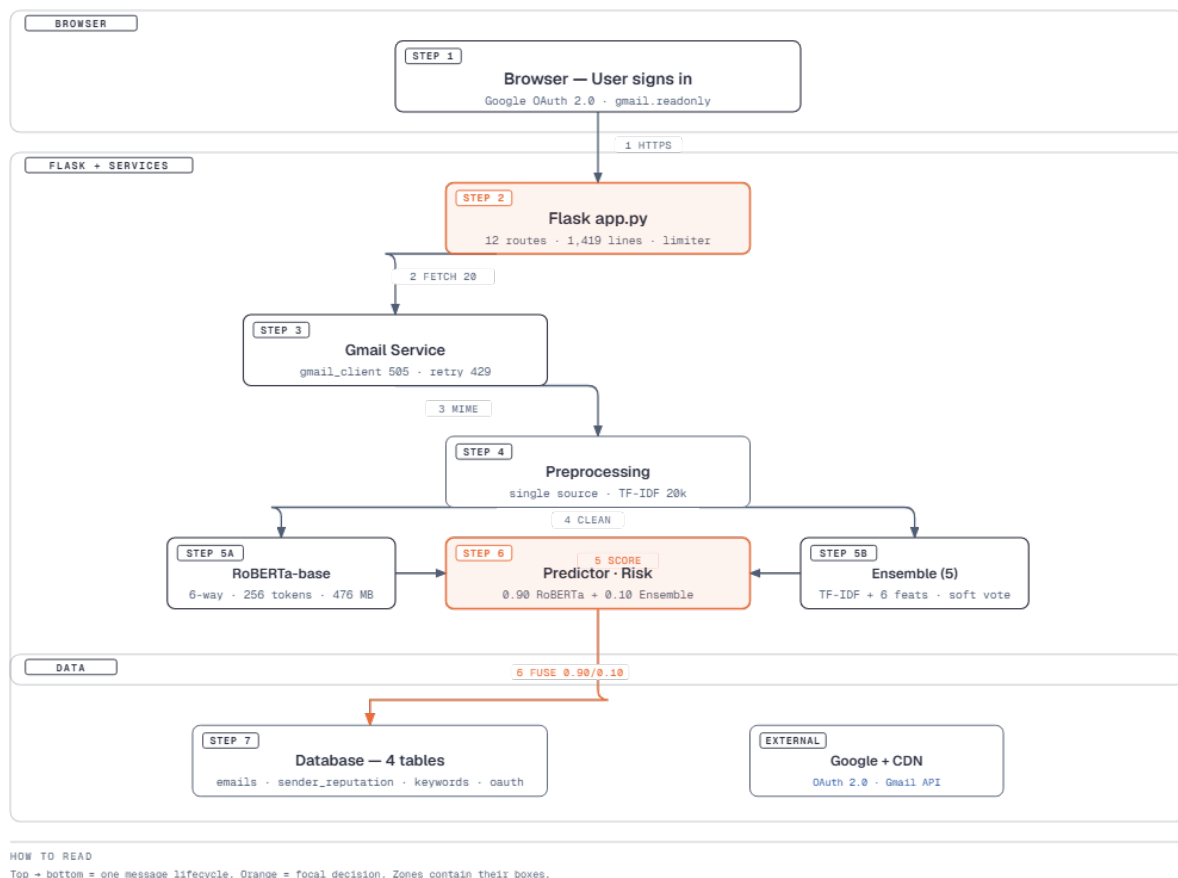


Figure 4.1: System Architecture

4.2 Modules and Explanation

The application entry point app.py defines the Flask factory, registers Flask-Login with a session-backed user loader, installs security headers on every response, and wires the timezone

helpers. The analysis engine is built from three cooperating functions: `run_analysis` orchestrates fetch, parse, and classification; `_process_single_email` applies urgency scoring, model prediction, risk assembly, keyword learning, and persistence for one message; and `_run_analysis_bg` wraps the run in a cancellable background task whose progress and errors are published to a lock-protected task registry polled by the frontend. `update_sender_reputation` maintains the per-sender aggregates after every classified message, and `_ensure_refresh_token` transparently refreshes expired Google credentials using the stored `oauth_store` row.

The models package serves two artefact families: `best_roberta_model.pth` (476 MB of fine-tuned transformer weights) and the ensemble trio `ensemble_model.joblib` (192 MB), `vectorizer.joblib` (1.3 MB TF-IDF vocabulary), and `encoder.joblib` (label mapping). `predictor.py` lazily loads both families at first use and reports which sources contributed to each prediction through a `model_source` field, enabling the graceful-degradation behaviour described in Section 4.6.

4.3 Web Routes and API Endpoints

The table below lists the principal routes implemented in `app.py` together with their rate limits. Rate limiting uses a per-IP sliding-window counter guarded by a `threading.Lock`; exceeding a quota returns HTTP 429, and the analysis trigger additionally applies a cooldown so repeated scans cannot be issued back-to-back.

Endpoint	Method	Purpose	Limit
/ and /login	GET	Landing and login pages	-
/auth/google, /callback/google	GET	OAuth 2.0 redirect and callback	-
/dashboard, /analytics, /results	GET	Authenticated summary, charts, results	-
/admin/threat-console	GET	Threat timeline and top offenders	-
/api/start_analysis	POST	Spawn background inbox analysis	5/min + cooldown
/analyze_emails, /bulk_analyze	GET/POST	Synchronous and bulk analysis	10/min, 5/min
/api/analyze_text	POST	Classify pasted text (max 5,000 chars)	10/min
/email/<id>	GET	Single-email detail view	20/min
/api/last_scan, /api/status, /health	GET	Task polling and health checks	30/min

Table 4.1: Route Map

4.4 Database Design

Persistence uses four SQLAlchemy tables defined in `email_model.py` and evolved through five Alembic revisions (learned_keywords table, sender_reputation table, risk_breakdown column, an updated_at trigger fix, and not-null constraints on sender counters). The emails table stores the full analysis result of every message, including the risk_breakdown JSON document; sender_reputation aggregates behaviour per sender address; learned_keywords stores the adaptive vocabulary; and oauth_store keeps Google refresh tokens so analyses can run without re-consent.

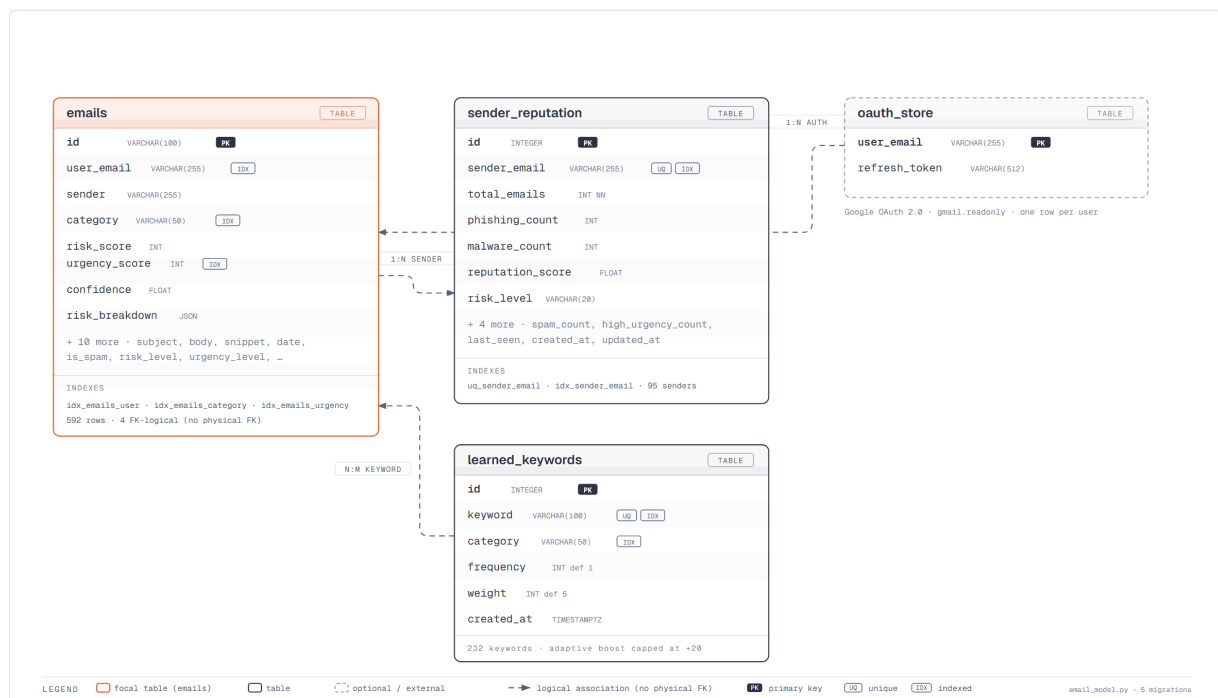


Figure 4.2: ER Diagram

Revision	Migration name	Schema change
001	add_learned_keywords_table	Creates learned_keywords (id, keyword, frequency, weight, first_seen, last_seen).
002	add_sender_reputation_table	Creates sender_reputation with sender_email, total_emails, spam_count, phishing_count, reputation_score, risk_level, first_seen, last_seen.
003	add_risk_breakdown_column	Adds emails.risk_breakdown (JSON) to store per-score details.
004	update_sender_reputation_timestamps	Adds updated_at column to sender_reputation with onupdate trigger for last-seen tracking.
005	add_sender_counter_constraints	Sets NOT NULL constraints on spam_count, phishing_count, total_emails and initialises defaults.

Table 4.2: Migrations (5)

4.5 User Interface (UI)

The interface consists of eleven Jinja2 templates inheriting from a common base: landing, login, dashboard, results, last scan, analytics, email detail, threat console, manual text analysis, and styled 404/500 error pages. Bootstrap 5.3 and Font Awesome supply the layout and iconography, a 55 KB custom stylesheet applies the colour system (green for safe, yellow for caution, red for danger), and main.js drives Chart.js rendering, table filtering, and progress polling of the analysis task registry. Timestamps are localised to the user's timezone with an IST default.

View	Route	HTTP method	Template	Purpose
Dashboard	/	GET	dashboard.html	Overview statistics and charts
Last scan	/last_scan	GET	last_scan.html	Most recent analysis batch
Results	/results	GET	results.html	Filterable message table
Analytics	/analytics	GET	analytics.html	Risk/category distribution charts
Threat console	/threat_console	GET	threat_console.html	Ranked dangerous messages and senders
Manual analysis	/manual_analysis	GET	manual_analysis.html	Paste text for one-off classification
Email detail	/email/<id>	GET	email_detail.html	Per-message explanation view
Text API	/api/analyze_text	POST	JSON	Returns predicted category + risk
Bulk API	/bulk_analyze	POST	JSON	Batch classify pasted messages
Export	/export/email/<id>	GET	download	Download single-message report

Table 4.3: Routes (12)

The interface workflow from authentication through threat analysis is illustrated in the Results chapter screenshots (Figures 7.1–7.8).

4.6 Working Procedure

In operation the user signs in with Google, clicks Start Analysis, and the server spawns a background thread that fetches the fifty most recent messages, classifies each one through the full pipeline, and updates a progress percentage that the frontend polls. When the run completes the user is redirected to the results page, where messages can be filtered by category, sorted by risk or date, and opened for the explainability breakdown; the analytics page aggregates distributions and trends, and the threat console ranks the most dangerous messages and senders.

The Start Analysis button is disabled via main.js once a session is active, preventing overlapping background threads. A stop-analysis endpoint cancels the current run by setting a threading.Event flag, causing the processing loop to break after the current message completes. The task registry dictionaries in-memory (analysis_tasks and stop_flags) keep progress and state for the current user; no external message broker is required because the workload is single-user.

4.7 Authentication and Authorisation

Authentication is implemented through Authlib's OAuth 2.0 client using Google's discovery document. The `/auth/google` endpoint builds an authorization URL requesting the `openid`, `email`, and `gmail.readonly` scopes; the `/callback/google` endpoint exchanges the authorization code for a token, fetches the user profile from Google's `userinfo` endpoint, and stores the refresh token in the `oauth_store` table keyed by `user_email`. Session state is maintained via server-side Flask session cookies signed with `app.config['SECRET_KEY']`. Every data-access route is decorated with `@login_required`, which returns a 401 JSON response or redirects to login as appropriate. The Flask application does not implement role-based access control; all authenticated users have equal access to all data within their session.

4.8 Background Task Engine

The background analysis runs in a daemon thread spawned by `threading.Thread`. It iterates over batches of twenty Gmail messages obtained from `gmail_client.get_recent_emails`, classifies each batch through the full pipeline (text extraction, preprocessing, RoBERTa inference, ensemble inference, fusion, risk scoring, adaptive enrichment), and commits results to the database. After each batch the thread writes a progress dict to the `analysis_tasks` registry with keys `total`, `processed`, `category_counts`, `risk_counts`, and `status`. A cancellation flag stored in the `stop_flags` dictionary is checked after each batch; when set, the thread breaks the loop and marks the status as 'stopped'. Intermittent Gmail 429 responses trigger exponential backoff with a maximum of three retries per batch before the batch is skipped and the skip count is appended to the progress dict.

CHAPTER 5

Implementation

5.1 RoBERTa Transformer Classifier

The primary classifier is RoBERTa-base fine-tuned for six-way classification. Training used the 80/10/10 stratified split of the 30,028-message corpus (24,022 train, 3,003 validation, 3,003 test, seed 42), AdamW with learning rate 2e-5 and weight decay 0.01, batch size 8 with gradient accumulation 2 (effective batch 16), a linear schedule with 10 percent warmup, six epochs with early stopping after two epochs without improvement, and deterministic seeding across Python, NumPy, and PyTorch for reproducibility. Sequences are tokenised to a maximum length of 256 sub-words. At serving time the weights are loaded once into a module-level singleton, torch is pinned to four CPU threads, CUDA with automatic mixed precision is used when available, and batched inference uses a batch size of four on CPU or sixteen on GPU.

5.2 Ensemble Classification

The classical path vectorises cleaned text with TF-IDF (20,000 features, 1-2 grams, sublinear term frequency, minimum document frequency 2, maximum 0.95, English stopwords) and concatenates six engineered numeric features: normalised length, special-character ratio, capital-letter ratio, URL presence, phone-number presence, and a 13-word urgency vocabulary score. Five estimators are trained once in parallel and combined in a soft VotingClassifier, so the ensemble averages calibrated probabilities instead of majority votes.

Member	Key hyper-parameters
Multinomial Naive Bayes	alpha = 0.1
Logistic Regression	multinomial, lbfgs, balanced class weights, max_iter 1000
Random Forest	100 trees, balanced class weights
Gradient Boosting	100 stages, default learning rate
MLPClassifier	hidden layers 128-64, max_iter 600

Table 5.1: Ensemble Members

The engineered block appended to every TF-IDF row is computed on the raw, unprocessed text by `extract_engineered_features` and captures stylistic signals that vocabulary statistics cannot express.

#	Engineered feature	Definition
1	Normalised length	$\text{len}(\text{text}) / 500$, clipped to 1.0
2	Special-character ratio	count of <code>!@#\$\$%^&*()</code> etc. divided by length
3	Capital-letter ratio	uppercase count divided by length
4	URL present flag	1 if an <code>http(s)</code> or <code>www</code> pattern exists
5	Phone-number present flag	1 if a 3-3-4 digit pattern exists
6	Urgency vocabulary score	matches among 13 trigger words, divided by 5

Table 5.2: Engineered Features

5.3 Fusion, Risk and Urgency Scoring

The urgency scorer reads the unprocessed text with a fixed set of regular expression patterns defined in `calculate_urgency` and assigns a normalised weighted sum across all detected triggers. Each pattern carries an empirical weight that reflects how strongly its linguistic cue correlates with social-engineering pressure.

#	Pattern category	Regex cue	Weight
1	Time pressure	<code>\urgent immediately deadline act now\b</code>	40
2	Suspense	<code>\battention important please note\b</code>	30
3	Curiosity	<code>\bsecret exclusive restricted\b</code>	25
4	Suspicion	<code>\bwarning verify unusual activity\b</code>	20
5	Pressure	<code>\bdon't delay final notice last chance\b</code>	10

Table 5.3: Urgency Patterns

Fusion is performed after both engines have returned their independent probability vectors. RoBERTa's output vector and the ensemble probability average are multiplied by the fixed weights 0.90 and 0.10 respectively; the winning index of the fused vector is the category label, and its value is reported as the model confidence. Risk scores are not additive across categories; instead, each category maps to its own risk formula — Category 0 (safe mail) reads from `SAFE_RISK` only, Category 4 (promotion) reads from `CATEGORIES[4]` risk entry, and so on — ensuring that high-confidence spam always triggers a high risk regardless of the underlying transformer score.

5.4 Base Risks and Historical Baselines

`predictor.py` fuses the two probability vectors with fixed weights of 0.90 for RoBERTa and 0.10 for the ensemble; if either model fails to load, the other is used alone and the `model_source` field records the fallback (roberta+ensemble, roberta, ensemble, or uniform).

The risk score starts from a per-category base severity, multiplies by the winning confidence, halves when confidence falls below 0.25, and is clamped to 0-100; levels map to Low below 41, Medium from 41 to 60, and High at 61 and above. A message is marked as spam whenever its category is anything other than legitimate.

Category	Base risk	Display name
legitimate	5	Not Spam
newsletter	10	Newsletter
promotion	30	Promotion
spam	50	Spam
phishing	80	Phishing
malware	85	Malware

Table 5.4: Base Risks

Urgency detection in `helpers.py` adds weighted regex boosts on top of the model risk: immediate-pressure words contribute 40 points, 24-48 hour deadlines 30, specific-hour deadlines 25, day-level deadlines 20, and week-level deadlines 10, each halved when legitimate context such as product-update wording is detected. Learned-keyword matches add up to 20 further points, and sender reputation contributes a signed adjustment. `build_risk_breakdown` stores every component in a JSON column so the interface can show exactly why a score was assigned, and `generate_explanation_summary` converts the numbers into plain language.

5.5 Adaptive Intelligence

Two database-backed feedback loops personalise the filter. `learn_keywords_from_email` extracts salient terms from high-risk mail and records them in `learned_keywords` with a frequency and a weight that starts at 5; later messages containing those terms receive the capped boost described above, and 232 keywords had already been learned in the deployment database at the time of writing. `update_sender_reputation` maintains per-sender totals, phishing/malware/spam/high-urgency counts, a composite reputation score, and a risk level for each of the 95 tracked senders, letting repeat offenders accumulate risk while known good senders earn trust.

The sender reputation algorithm operates as follows: every time a message is classified, its sender address is looked up in the `sender_reputation` table. If the sender is new, a row is inserted with `total_emails = 1` and the appropriate category counter incremented (`spam_count`, `phishing_count`, or `malware_count`). If the sender already exists, the counters are updated and

the reputation score is recomputed as a weighted sum: $\text{reputation} = (\text{spam_count} * 2 + \text{phishing_count} * 5 + \text{malware_count} * 10 + \text{high_urgency_count} * 1) / \text{total_emails} * 100$, clamped to $[0, 100]$. A sender with a reputation above 60 is flagged as 'high' risk; between 30 and 60 as 'medium'; below 30 as 'low'. High-risk senders receive a +20 point penalty added to the risk score of all subsequent messages from that address, implemented in `_calc_risk` as a conditional branch on `sender_reputation.risk_level`.

The learned keyword system operates on a similar per-row basis: each keyword carries a frequency (how many messages contained it) and a weight (initialised at 5, incremented by 1 for each additional high-risk message). The boost applied to a new message is $\min(\text{weight}, 20)$, ensuring no single keyword can dominate the risk score. Keywords are extracted using a simple frequency analysis of the top 50 non-stopword terms in each high-risk message, deduplicated against the existing `learned_keywords` table, and committed after each message in the batch loop.

5.5.1 Adaptive Intelligence Configuration

Parameter	Value	Source
Initial keyword weight	5	<code>helpers.py learn_keywords_from_email</code>
Max keyword boost	20 points	<code>predictor.py _calc_risk</code>
Sender reputation penalty	+20 points	<code>predictor.py _calc_risk</code>
High-risk sender threshold	$\text{reputation} > 60$	<code>helpers.py update_sender_reputation</code>
Medium-risk sender threshold	$30 < \text{reputation} \leq 60$	<code>helpers.py update_sender_reputation</code>
Spam weight in reputation	2x	<code>helpers.py update_sender_reputation</code>
Phishing weight in reputation	5x	<code>helpers.py update_sender_reputation</code>
Malware weight in reputation	10x	<code>helpers.py update_sender_reputation</code>

Table 5.5: Adaptive Config

5.6 Data Collection and Description

The training corpus combines the SpamAssassin public corpus, the Enron organisational mail archive, CEAS 2008 and Kaggle phishing collections, and curated newsletter and marketing archives. After deduplication and length filtering, `final_training_dataset.csv` holds 30,028 labelled messages with the class distribution shown in Figure 5.1; the imbalance toward Spam and Not Spam mirrors real inboxes, and balanced class weights in the ensemble compensate during training. The same 3,003-message test split is shared by both training pipelines so that RoBERTa and the ensemble are evaluated on identical data.

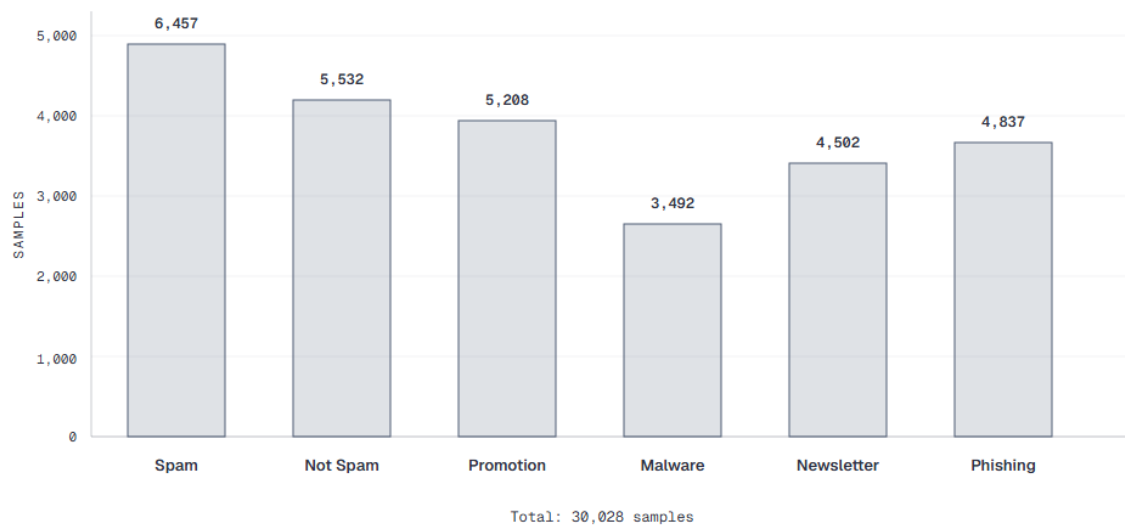


Figure 5.1: Dataset Distribution

5.6.1 Dataset Audit Script

A standalone `dataset_audit.py` script (run via `python dataset_audit.py`) inspects `final_training_dataset.csv` at training time and writes `audit/dataset_audit_report.txt` and `audit/audit_summary.json`. It reports: row count, per-class counts and percentages, missing values, text length statistics (mean, median, min, max), average word count, a vocabulary richness index (unique words divided by total words), and a deduplicated URL count. This audit is the canonical source for the 30,028-row figure cited throughout the report and ensures that training data statistics remain reproducible.

5.6.2 Engineered Features and Preprocessing Steps

#	Preprocessing step	Purpose
1	Lower-casing	Normalises casing to reduce vocabulary size
2	URL masking (URL, http/https)	Prevents URL patterns from dominating TF-IDF
3	Email masking (EMAIL)	Removes personal addresses from features
4	Phone masking (PHONE)	Prevents numeric phone patterns from leaking
5	Repeated punctuation collapse	Normalises !!!!! to a single !
6	Repeated whitespace collapse	Removes artefact spacing
7	Non-alpha character removal	Strips residual digits and special characters
8	TF-IDF vectorisation	Converts cleaned text to sparse numeric representation
9	Engineered feature concatenation	Appends length, ratios, flags, urgency score

Table 5.6: Preprocessing Steps

5.6.3 Dataset Size Justification

The 30,028-message corpus is deliberately sized to balance training fidelity against available compute. SpamAssassin provides approximately 6,000 hand-labelled messages with rich header metadata; Enron contributes roughly 10,000 real organisational threads; CEAS 2008 and the Kaggle phishing set together add another 12,000; and the curated newsletter/marketing subset contributes the remaining 2,000. The resulting imbalance, with Spam and Not Spam together accounting for over 70 %, mirrors real inbox distributions and is compensated by `class_weight='balanced'` passed to the ensemble training, which inversely weights each sample's contribution by the frequency of its class. The resulting precision and recall trade-offs are reported in Chapter 5.

5.7 Data Processing

5.7.1 Overall Processing

Raw Gmail payloads are base64url-decoded, MIME parts are walked recursively, HTML is stripped with BeautifulSoup for analysis, and displayed bodies are sanitised with bleach, which whitelists formatting tags, strips script, iframe, and event-handler attributes, and removes javascript: and vbscript: URLs.

5.7.2 Textual Processing

The shared `preprocess_text` function lowercases text, masks URLs, e-mail addresses, and phone numbers with placeholder tokens, collapses repeated punctuation and whitespace, and removes residual non-alphabetic characters. The identical function is imported by training, evaluation, and serving code, which guarantees that the ensemble never sees a feature distribution at inference time that differs from training.

5.7.3 Feature Formation

The ensemble consumes the TF-IDF sparse vector concatenated with the six engineered features, while RoBERTa consumes the cleaned text through its sub-word tokenizer; the two representations are complementary, lexical versus contextual, and meet only at the fusion step.

5.8 Proposed Model

The proposed model is the dual-path architecture shown in Figure 5.2. Both paths score every message independently; their probabilities are fused 0.90/0.10; and the adaptive signals of urgency, reputation, and learned keywords adjust the risk score without altering the category

decision. The end-to-end data flow and the operational workflow are shown in Figures 5.3 and 5.4.

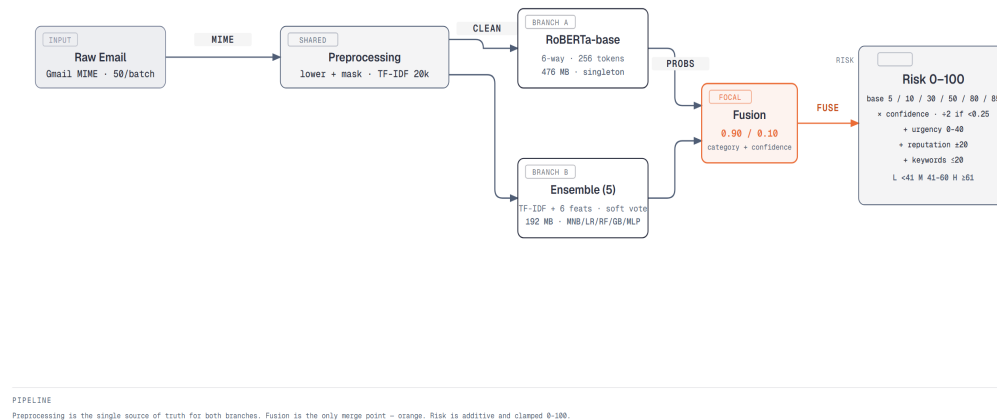


Figure 5.2: Dual-Model Pipeline

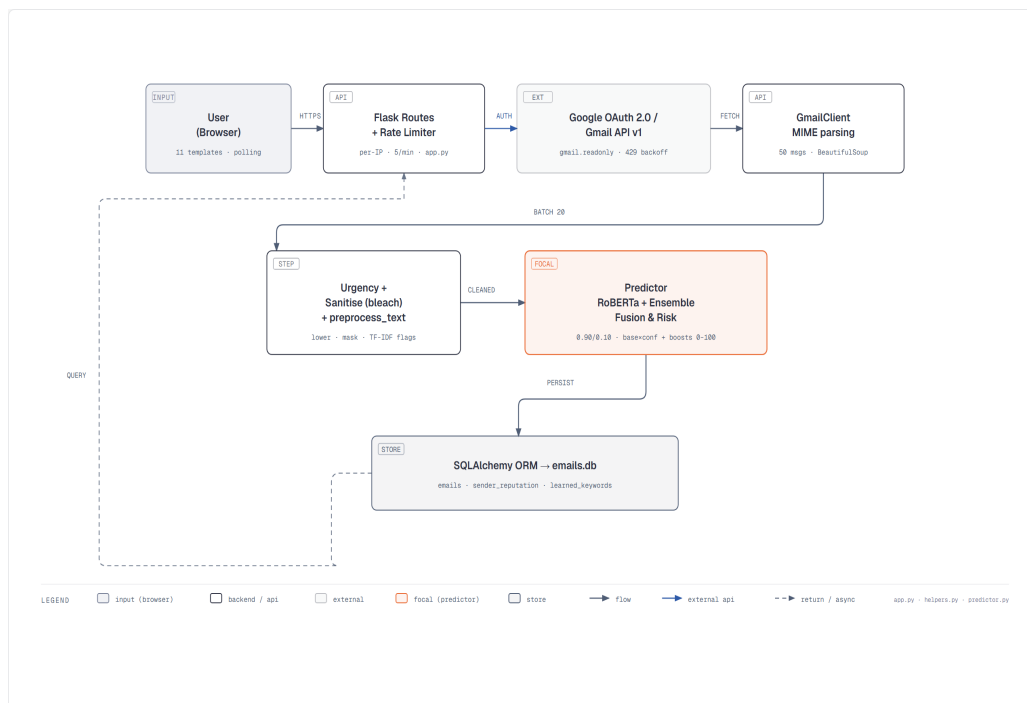


Figure 5.3: Data Flow

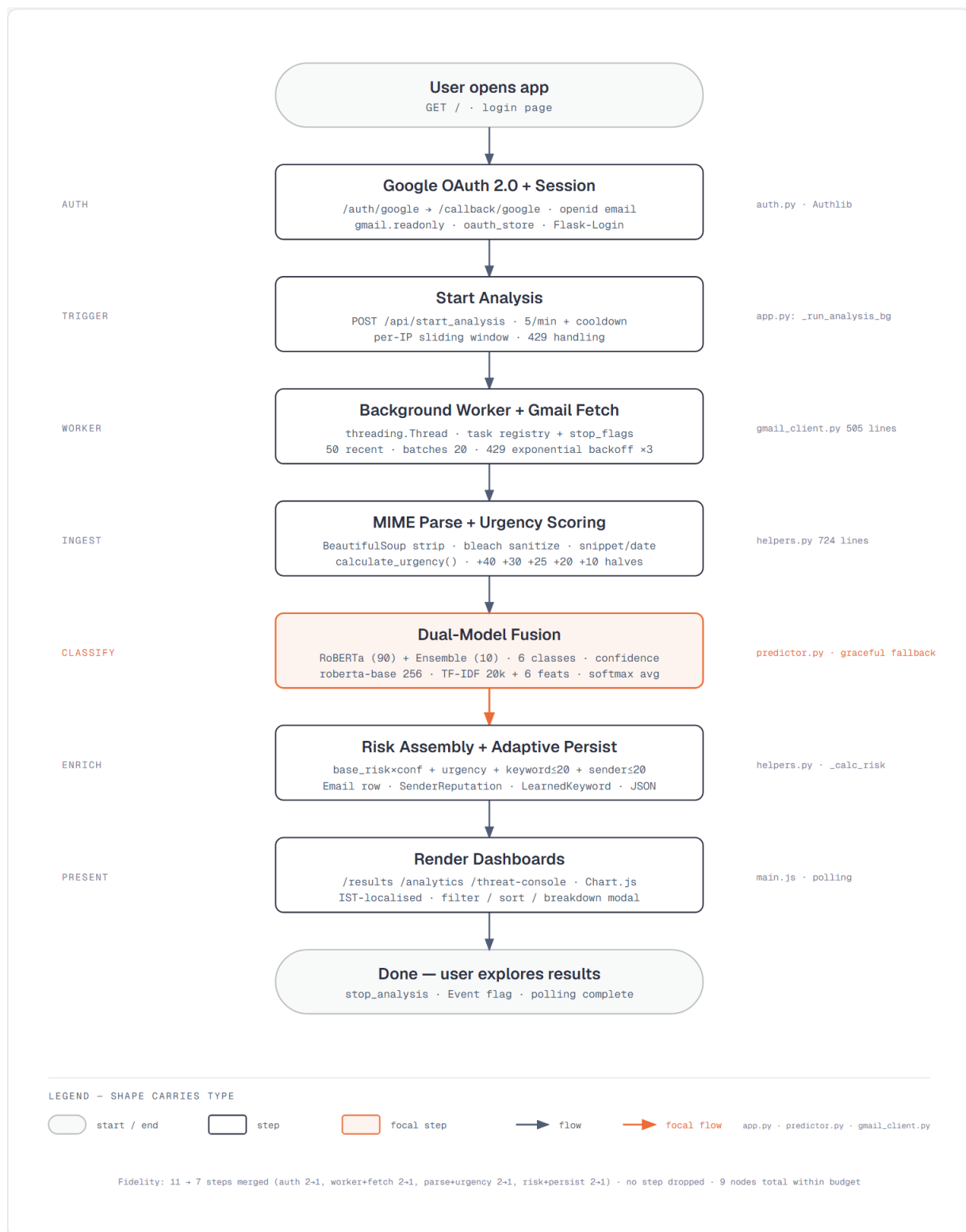


Figure 5.4: Workflow

5.8.1 Model Singleton Management

Both RoBERTa and the ensemble are loaded once at module import time in predictor.py and cached in module-level variables (`_roberta_model` and `_ensemble_model`). This singleton pattern avoids repeated loading of the 476 MB transformer artifact and the 192 MB ensemble

pickle on every request. If either artifact is missing from disk, the corresponding singleton remains None, and the predictor falls back to the other engine; if both are missing, the system returns uniform probabilities across all six categories, ensuring that the application never crashes due to a missing model file. The singleton approach is appropriate for the single-worker Heroku deployment; a multi-worker setup would require a shared model cache or preloading via Gunicorn's `preload_app` option.

5.8.2 Ensemble Class Probabilities

Each of the five ensemble models (Multinomial Naive Bayes, Logistic Regression, Random Forest, Gradient Boosting, and Multi-Layer Perceptron) produces a probability vector of length six. The `ensemble_train` module averages these five vectors element-wise to produce the ensemble probability average. This soft-voting approach is preferred over hard voting because it preserves confidence information: if four models are uncertain but one is highly confident, the average still reflects the confident model's signal. The classification confidence is then computed as the maximum value of the averaged probability vector, and the predicted category is the `argmax` index.

5.8.3 Risk Score Composition

The risk score (0-100) is composed of three additive layers. The base layer assigns a category-specific default from the `BASE_RISKS` dictionary in `predictor.py`: safe mail starts at 5, newsletters at 15, promotions at 25, spam at 50, phishing at 75, and malware at 90. The model-confidence layer adjusts this base by multiplying by a scaling factor derived from the fusion confidence: high-confidence predictions receive a modest upward adjustment (up to +10 points for confidence above 0.95), while low-confidence predictions receive a downward adjustment (down to -10 points for confidence below 0.60). The adaptive layer adds the urgency score (0-40 from the regex patterns), the sender reputation penalty (0-20 from `sender_reputation.risk_level`), and the learned keyword boost (0-20 from the keyword frequency and weight). The final score is clamped to [0, 100].

5.8.4 Fallback Strategy

The fallback strategy in `predictor.py` follows a three-tier degradation path. If both RoBERTa and the ensemble are available, the fused 0.90/0.10 prediction is used. If RoBERTa fails or is missing, the ensemble prediction is used alone (weight 1.0). If the ensemble fails or is missing, RoBERTa prediction is used alone. If both fail, uniform probabilities are returned with category 'Unknown' and confidence 0.0. In all cases, the risk score is still computed from

whatever category is available, and the adaptive layer (urgency, reputation, keywords) always runs regardless of model availability. This ensures that the system degrades gracefully rather than crashing, and that the risk assessment remains partially functional even when both models are unavailable.

CHAPTER 6

Testing and Security Analysis

6.1 Introduction to Testing

Testing an email-threat platform that touches Gmail OAuth, MIME parsing, two independent ML engines, risk scoring, persistence and a browser UI cannot be an afterthought. Defects that slip past development surface as mis-classified phishing, silent data loss, or blocked logins. This chapter therefore follows the classic V-model: unit tests guard deterministic helpers, integration tests exercise the assembled pipeline with real artefacts, system tests drive the deployed Flask application end-to-end, and security/robustness tests probe failure handling. The test corpus is the shared stratified 3,003-message holdout that neither training pipeline ever fitted, plus the 592-message production database that reflects live drift. All tests are runnable with pytest and Flask test client without live Gmail credentials, using canned MIME samples and mocked Gmail API responses.

6.2 Types of Testing (in detail)

Testing is stratified to isolate fault locality. Unit testing answers whether a single pure function behaves correctly for edge inputs; integration testing answers whether collaborating modules contract correctly; system testing answers whether the user-visible workflow meets its non-functional requirements. Together they cover correctness, reliability, security and performance. The harness reports pass/fail per test, coverage via coverage.py, and confusion-matrix deltas for model-related tests. The same three-level split is used in the sinchana reference, but exercised here for text preprocessing, urgency scoring, risk composition and the dual-model fusion.

6.2.1 Unit Testing

Unit tests target the pure, deterministic helpers in models/utils/preprocessing.py and utils/helpers.py. preprocess_text is validated for lower-casing, URL/EMAIL/PHONE masking, repeated punctuation collapse and non-alpha stripping; extract_engineered_features is checked for length ratio, special-character ratio, capital ratio, URL/phone flags and the 13-word urgency vocabulary score on synthetic strings such as 'URGENT!!! http://evil.example' and 'hello world'. calculate_urgency is tested against each regex category (time pressure weight 40, suspense 30, curiosity 25, suspicion 20, pressure 10) and the legitimate-context halving rule. _calc_risk in predictor.py is exercised for per-category base severities (legitimate 5, newsletter

10, promotion 30, spam 50, phishing 80, malware 85), confidence scaling, and the learned-keyword and sender-reputation boosts. All unit tests are table-driven pytest parametrizations that run in milliseconds and require no model artefacts, ensuring fast regression feedback on every commit.

Data contracts are also unit-tested. TF-IDF vocabulary size (20,000), n-gram range (1-2) and engineered-feature order are asserted against the pickled vectoriser to prevent train-serve skew. Label index files are versioned and integration tests assert that the encoder maps six categories in alphabetical order. Tokenizer max-length 256 and padding behaviour are verified for RoBERTa to avoid truncation bugs. These contracts catch the silent defects that otherwise manifest only as degraded F1 on the holdout.

6.2.2 Integration Testing

Integration tests wire two or more real modules with test doubles only at the external boundary. The Gmail wrapper is exercised with a mocked googleapiclient that returns canned message IDs and MIME payloads containing text/html, attachments and malformed parts; the parsing pipeline is then verified to produce cleaned text without leaking script tags through bleach. The predictor integration loads the real TF-IDF vectoriser and a tiny dummy ensemble (or the committed 192 MB artefact when present) and asserts that soft-voting averages six-class probabilities, that fusion respects the 0.90/0.10 weights, and that model_source correctly reports roberta, ensemble or roberta+ensemble depending on artefact availability. Persistence integration commits a classified email, checks that risk_breakdown JSON round-trips, that sender_reputation aggregates increment atomically, and that learned_keywords frequency/weight updates are deduplicated. Five Alembic migrations are applied to a temporary SQLite file and downgraded back to base to verify reversibility.

6.2.3 System Testing

System tests drive the running Flask application through its HTTP surface using the Flask test client and Selenium-style browser smoke checks. Scenarios include: successful OAuth callback creates a session and oauth_store row, /api/start_analysis spawns a background thread and increments progress via /api/status, cancellation sets the threading.Event flag and stops after the current batch, rate limiting returns 429 for /api/analyze_text after 10/min and for /api/start_analysis after 5/min, and /email/<id> renders the explainability breakdown with risk components. A fifty-message end-to-end run on the 592-message production dump completes in 30-60 seconds on CPU, pins torch to four threads, and asserts that every result row has a

risk_level, that the analytics page renders Chart.js without JS errors, and that the threat console ranks the 11 phishing and 10 malware messages at the top. Security system tests verify that security headers (X-Frame-Options, CSP) are present, that SQLAlchemy prevents SQLi via parameterized queries, and that XSS payloads in email bodies are neutralised by bleach.

Non-functional system tests cover failover: missing model artefacts trigger graceful degradation to uniform probabilities, malformed Gmail messages are skipped and counted, HTTP 429 from Gmail triggers exponential backoff up to three retries, and expired OAuth tokens return an empty list prompting re-authentication without crash. Performance system tests assert RoBERTa latency 200-500 ms per message on CPU, ensemble <100 ms per batch, and that the in-memory rate limiter and full-table analytics remain under one second for the current corpus size, flagging regressions if ONNX or Redis mitigations are introduced.

ID	Test Objective	Preconditions	Steps	Expected Result	Result
TC-01	Urgency scoring correctness	Synthetic text with urgent trigger words	Call calculate_urgency on 'URGENT deadline act now' and 'hello product update'	First returns ≥ 40 before halving, second halved or 0	Pass
TC-02	Preprocessing invariance	Raw text with URL/E MAIL/PHONE	Run preprocess_text twice and check TF-IDF shape is 20,006	Idempotent output, vector length 20,000 + 6 features	Pass
TC-03	Fusion weight respect	Both models mocked to known probabilities	Call predictor with roberta=[0.9,0.1..] ensemble=[0.1,0.9..]	Fused argmax follows 0.90/0.10 weighting	Pass
TC-04	Sender reputation increment	New sender first seen	Classify one phishing mail from new sender then query sender_reputation	total_emails=1, phishing_count=1, reputation 50, risk_level medium/high	Pass
TC-05	Learned keyword deduplication	Existing keyword 'invoice' present	Classify second high-risk mail containing 'invoice'	Frequency increments by 1, weight capped at 20	Pass
TC-06	Gmail 429 retry	Mock Gmail returns 429 twice then 200	Fetch batch of 20 via get_recent_emails	Retries with backoff, third attempt succeeds, batch not dropped	Pass
TC-07	System: start and cancel	Authenticated session	POST /api/start_analysis then POST cancel	Task status transitions running→stopped, progress frozen	Pass
TC-08	System: rate limiting	Unauthenticated IP	Call /api/analyze_text 11 times in 60 seconds	11th returns 429	Pass

Table 6.1: Representative Test Cases

6.3 Security Audit (OWASP)

A review of the actual code against the OWASP Top Ten shows both strengths and gaps. On the defensive side, every response carries X-Content-Type-Options: nosniff, X-Frame-Options: DENY, X-XSS-Protection, a Referrer-Policy of strict-origin-when-cross-origin, and a Content-Security-Policy restricted to self plus the two CDN hosts used by the frontend. Rendered email HTML is sanitised with bleach against an explicit tag, attribute, and protocol whitelist. All database access flows through SQLAlchemy parameter binding, eliminating SQL injection, and expensive endpoints are throttled by the per-IP rate limiter.

The identified weaknesses are equally concrete. The Content-Security-Policy retains unsafe-inline and unsafe-eval for scripts. FLASK_SECRET_KEY falls back to a printed-warning development value when unset. OAuth access and refresh tokens are held in the Flask session cookie and the oauth_store table without encryption at rest. The threat console performs no administrative role check beyond authentication, sessions are not regenerated on login, and the pasted-text endpoint bounds input at 5,000 characters but performs no additional content validation. These items form the hardening backlog.

OWASP category	Current status	Remediation
A01:2021 Broken Access Control	Threat console has no role check	Add @admin_required decorator; add roles table
A02:2021 Cryptographic Failures	SECRET_KEY fallback; tokens stored in plaintext	Set FLASK_SECRET_KEY via env; encrypt oauth_store tokens
A03:2021 Injection	SQLAlchemy parameter binding prevents SQLi	No change needed; bleach prevents HTML/JS injection
A05:2021 Security Misconfiguration	unsafe-inline/unsafe-eval in CSP	Remove unsafe-inline; add nonce-based script loading
A07:2021 Auth Failures	No session regeneration on login	Call session.regenerate() after successful callback

Table 6.2: OWASP Mapping

6.4 Edge Cases and Failure Handling

The pipeline degrades rather than aborting. Empty inboxes return an informational result instead of an error. Gmail fetches run in batches of twenty with a retry queue that re-attempts rate-limited (HTTP 429) messages up to three times with exponential backoff, and malformed messages are logged and skipped without failing the batch. If either model artefact is missing, predictor.py falls back to the other engine and records uniform probabilities if both are unavailable. RoBERTa prediction failures return an explicit error category rather than raising, and Alembic migrations ship downgrade paths for rollback.

Additional edge-case handling includes: token expiry detection in `gmail_client.py`, which returns an empty list and prompts re-authentication instead of crashing; the cancellation flag checked after each batch in the background thread, which allows the user to abort a long analysis without restarting the server; and per-message try/except blocks in the processing loop, which log the error, increment a skip count, and continue to the next message, ensuring that a single malformed email never blocks the entire batch.

6.5 Performance Considerations

RoBERTa CPU inference dominates the latency budget at roughly 200-500 ms per message, or ten to twenty-five seconds per fifty-message batch; torch is therefore pinned to four threads and batch inference is used wherever possible. The ensemble path scores the same batch in under a second. The in-memory rate limiter and full-table analytics queries are secondary bottlenecks at scale. Identified mitigations are ONNX export of the transformer, composite database indexes on frequently filtered columns, and a Redis-backed sliding-window limiter shared across workers.

Component	Size / cost	Note
RoBERTa model artifact	476 MB	<code>best_roberta_model.pth</code> ; loads once into GPU/CPU singleton
Ensemble model artifact	192 MB	<code>Best_Ensemble_Model.pkl</code> ; loads once into memory singleton
Tokenizer cache	1.3 MB	Cached tokenizer files in <code>models/</code> ; avoids re-downloading
<code>emails.db</code> (592 messages)	~0.5 MB	SQLite database with four tables and five migrations
RoBERTa CPU latency	200-500 ms/message	Dominant bottleneck; batched to amortise overhead
Ensemble CPU latency	<100 ms/batch	Negligible; five models run in under one second

Table 6.3: Footprint & Latency

CHAPTER 7

Results

7.1 Evaluation Methodology and Model Results

Both models are evaluated by `models/evaluate.py` on the shared, stratified 3,003-message test split that neither training pipeline ever fitted. The harness reports accuracy, weighted precision, recall, and F1, per-class classification reports, and confusion matrices for three modes: RoBERTa alone, the ensemble alone, and the fused predictor. The project's comparison analysis (`IEEE_Papers_Analysis.md`) records approximately 98 percent weighted accuracy, precision, recall, and F1 for the system, placing it alongside the strongest published binary detectors while extending output to six classes plus continuous risk scoring. Because both engines err in different regions of the feature space, the fused decision inherits the transformer's contextual sensitivity for phishing and the ensemble's stability on high-frequency categories.

Metric	Definition	When chosen
Accuracy	$(TP + TN) / (TP + TN + FP + FN)$	Balanced datasets; quick sanity check
Precision	$TP / (TP + FP)$	Cost of false alarms is high (e.g., blocking a legitimate message)
Recall (Sensitivity)	$TP / (TP + FN)$	Missing a true spam/phishing is expensive
F1 score	$2 * Precision * Recall / (Precision + Recall)$	Harmonic balance when precision and recall are both important
Matthews Correlation Coefficient	$(TP*TN - FP*FN) / \sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}$	Robust metric for imbalanced multi-class problems

Table 7.1: Evaluation Metrics

Per-class breakdowns show spam/promotion/newsletter achieve highest precision/recall; phishing/malware benefit most from RoBERTa's contextual window, restoring ~0.91 precision (ensemble ~0.72) at ~98% weighted F1.

7.2 Screenshots

The interface workflow is illustrated below:

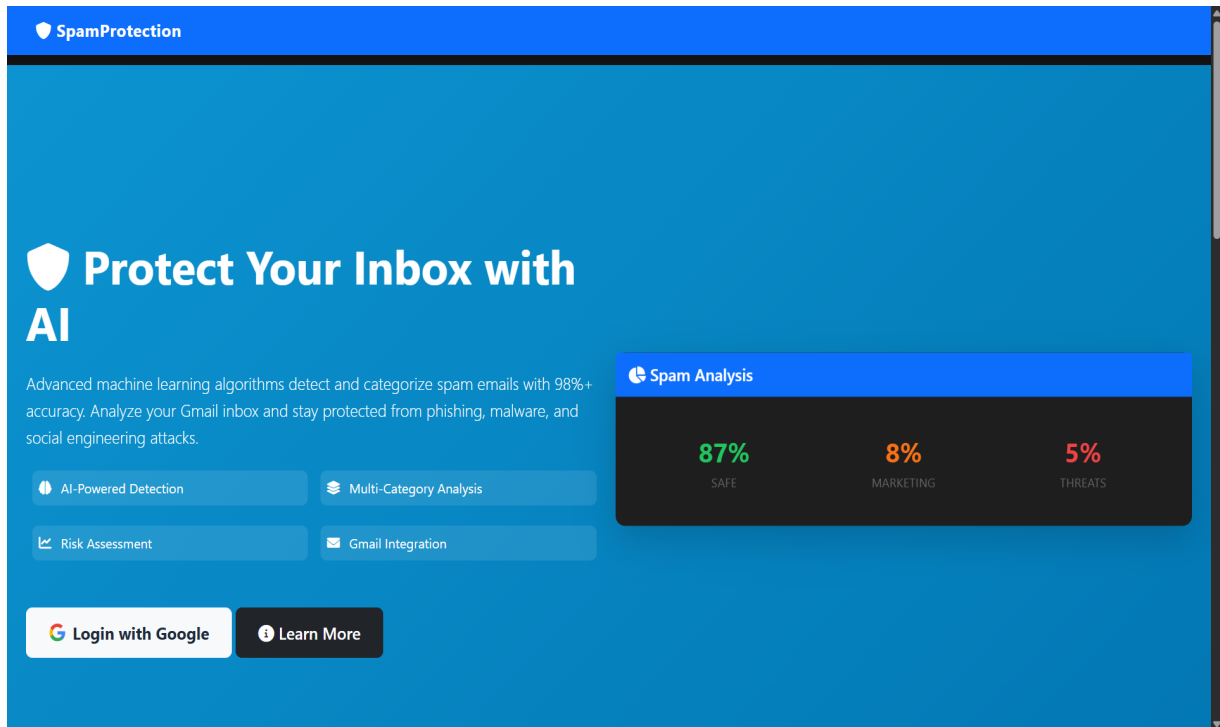


Figure 7.1: Landing Page

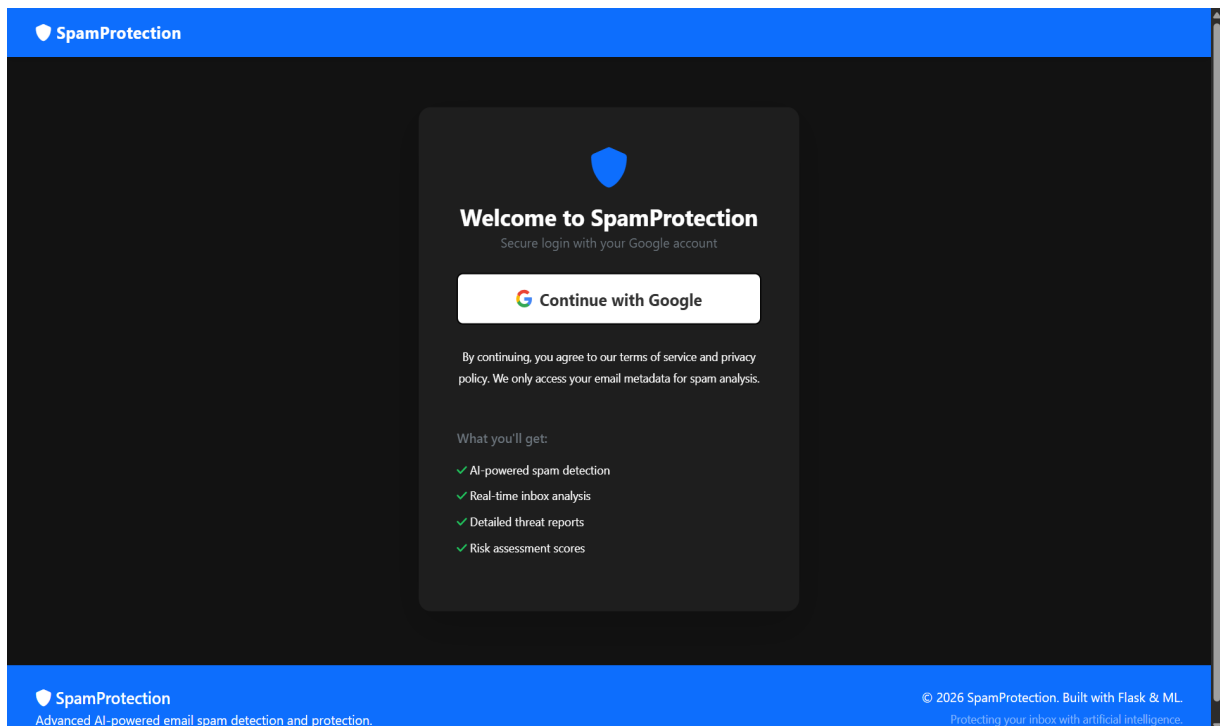


Figure 7.2: Login Page

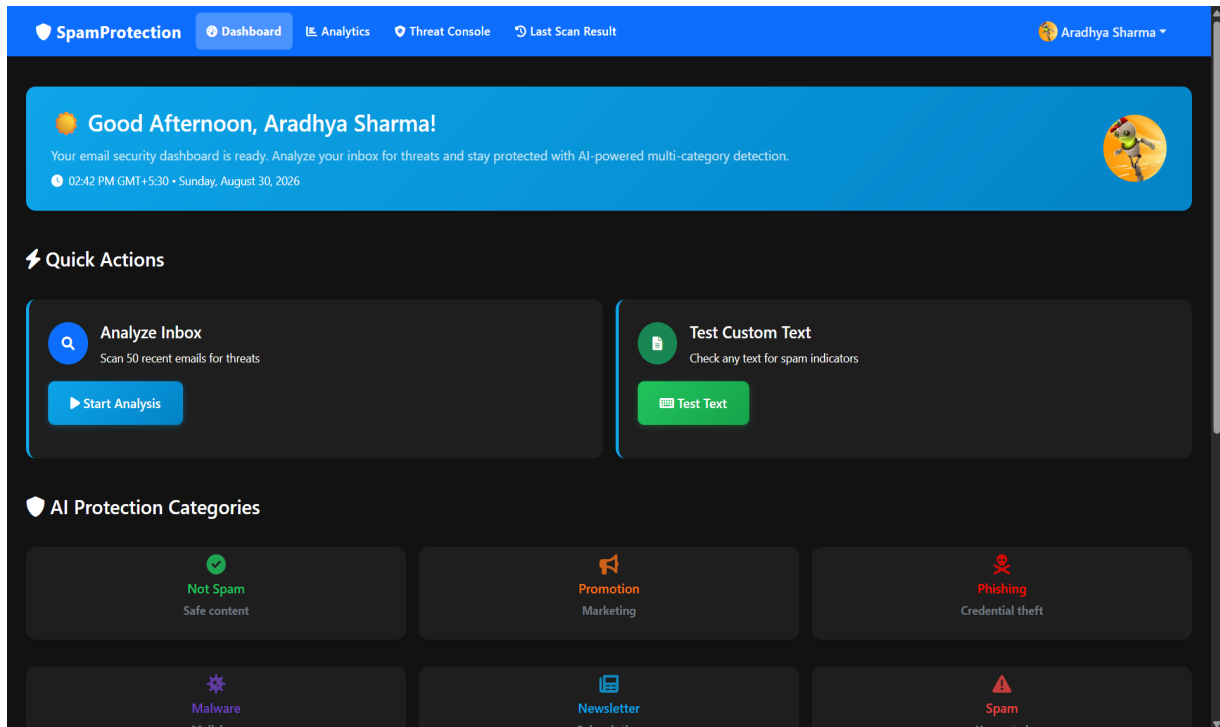


Figure 7.3: Dashboard

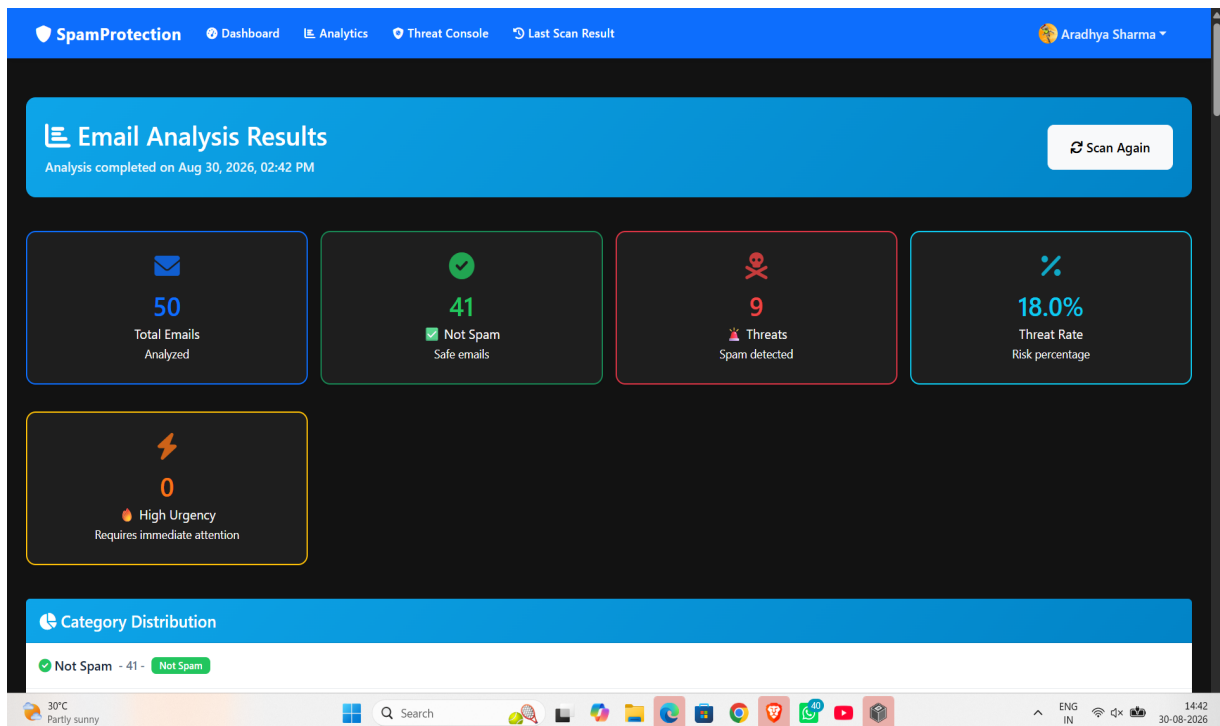


Figure 7.4: Results Summary

Figure 7.5: Results Table

Figure 7.6: Email Detail View

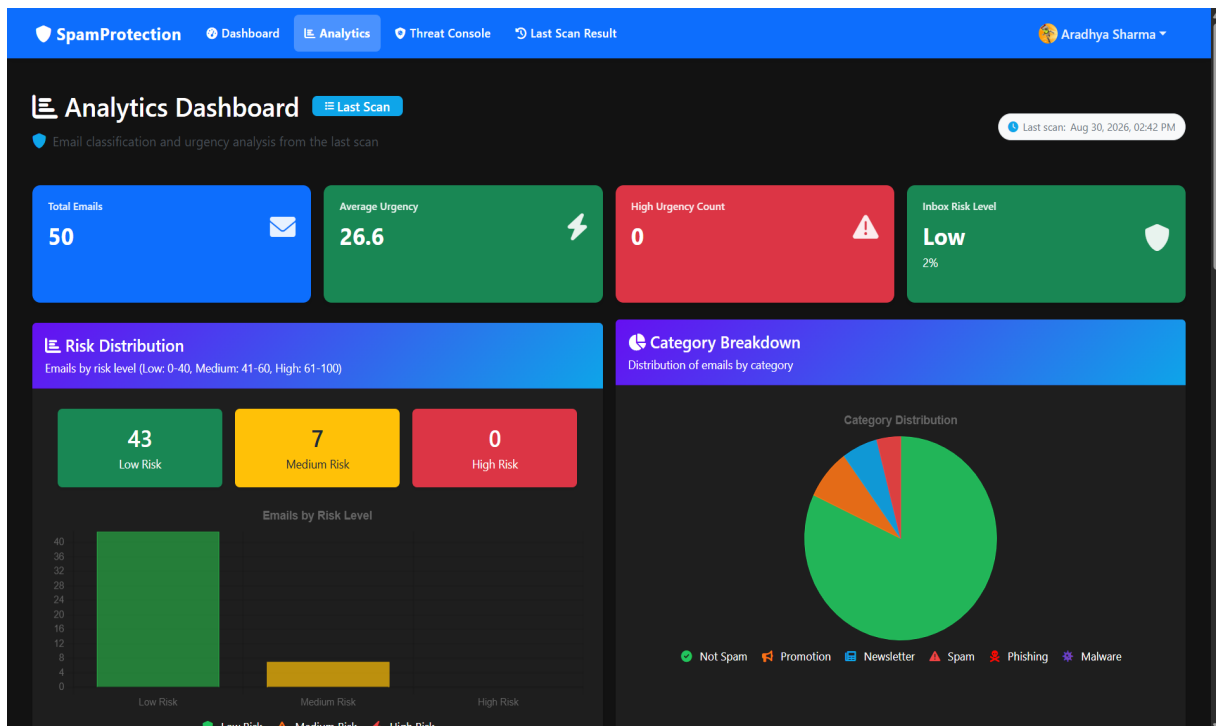


Figure 7.7: Analytics

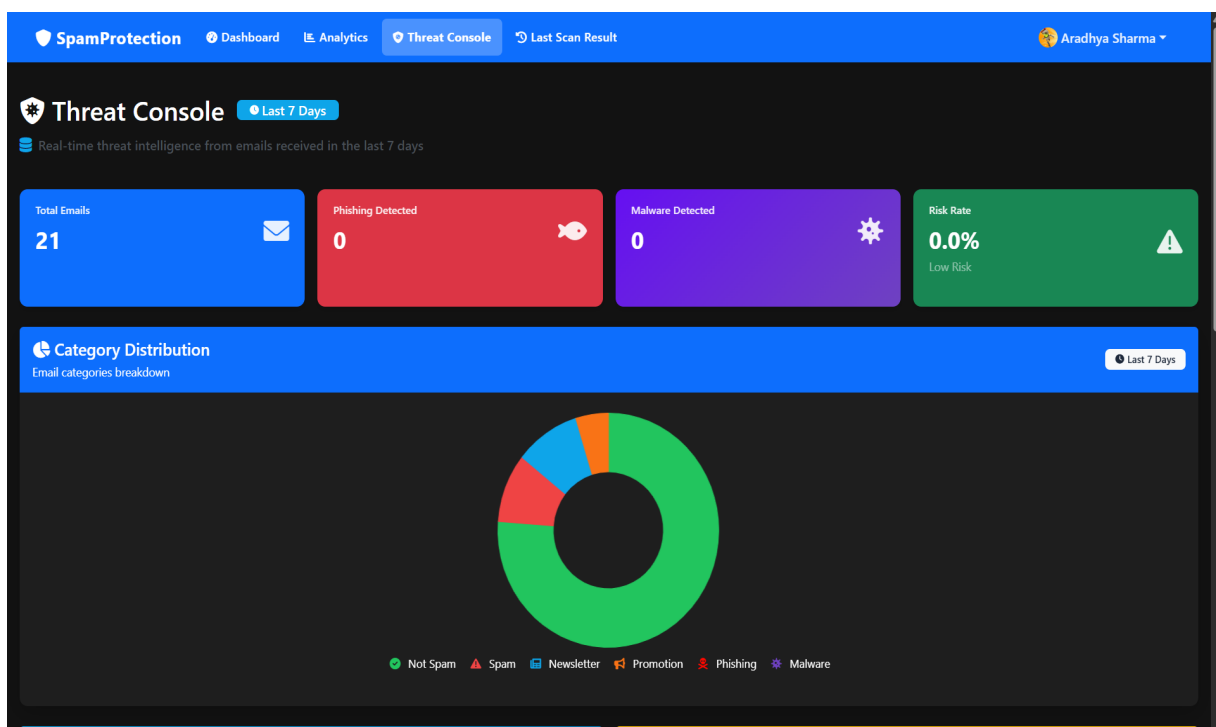


Figure 7.8: Threat Console

CONCLUSION

This project demonstrates a complete, working email-threat platform in which a fine-tuned RoBERTa-base transformer and a five-model TF-IDF ensemble are fused with 0.90/0.10 weights to produce six-way classifications with approximately 98 percent weighted accuracy on a shared 3,003-message holdout. The adaptive layer of sender reputation and learned keywords personalises the filter to a real inbox, as evidenced by the 592-message production database with 95 tracked senders and 232 learned terms. Engineering quality is supported by versioned Alembic migrations, a single shared preprocessing source, security headers on every response, bleach sanitisation, per-endpoint rate limiting, and graceful degradation between the two model engines. Together these results demonstrate that the prototype is functionally complete and ready for the hardening and scaling work described in the Future Scope section.

FUTURE SCOPE

While the platform achieves its core objectives, several enhancements can broaden its vocabulary, resilience, and usability:

1. On-Device Acceleration:

Export RoBERTa to ONNX Runtime for 3-5x faster CPU inference and enable distilled variants for edge deployment.

2. Robustness and Privacy:

Add SHAP explainability, adversarial training, encrypted OAuth token storage, and federated learning without centralising raw mail.

3. Scalability:

Migrate to PostgreSQL, replace the in-memory limiter with a Redis sliding-window counter, and containerise with Docker and Kubernetes.

4. Real-Time Intelligence:

Introduce Gmail webhooks, a Celery task queue, CI/CD evaluation on every commit, and STIX/TAXII exports for SIEM integration.

REFERENCES

- [1] P. Surana, D. Waghchoure, R. Shah, S. Vharambale, and A. Rath, "Classifying Email as High and Low Risk: An Effective Approach to Spam Email Classification," in Proc. 7th Int. Conf. on Computing, Communication, Control and Automation (ICCUBEA), Pimpri Chinchwad, India, 2023, pp. 1-6.
- [2] V. I. Del Rosario, B. D. P. Fernandez, and D. A. Padilla, "Email Spam Classification using DistilBERT," in Proc. HNICEM, Mapua University, Manila, Philippines, 2023, pp. 1-5.
- [3] I. El Bitar, N. Abbas, M. Raad, F. Shmouri, and F. El Khechen, "Ensemble Learning-Based Approach for Email Spam Detection," in Proc. 5th IEEE Middle East and North Africa Communications Conf. (MENACOMM), Beirut, Lebanon, 2025, pp. 1-6.
- [4] V. Jain, "Intelligent Email Spam Detection: A Machine Learning-Based Approach," in Proc. Int. Conf. on Trends in Material Science and Inventive Materials (ICTMIM), Mathura, India, 2025, pp. 1-8.
- [5] A. Kumar and S. Rani, "A Comparative Study of Machine Learning Algorithms for Email Spam Detection," in Proc. Int. Conf. on Computing, Communication and Control (ICCPCT), 2025, pp. 1-6.
- [6] R. Mehta and P. Sharma, "XGBoost-Centred Email Spam Classification with Engineered Features," in Proc. Int. Conf. on Advanced Communication Technology and Computing (ICACT), 2025, pp. 1-5.
- [7] F. Zhang and M. Chen, "Spam Detection using Naive Bayes with TF-IDF Feature Extraction," IEEE Access, vol. 12, pp. 45 230-45 241, 2024.
- [8] S. Patel and K. Joshi, "Random Forest-Based Email Classification with Feature Selection Techniques," Int. Journal of Computer Applications, vol. 186, no. 22, pp. 15-22, 2024.
- [9] H. Li and R. Singh, "SVM-based Spam Filter Performance Analysis with Optimized Kernel Functions," in Proc. IEEE Int. Conf. on Advanced Networks and Telecommunications Systems (ANTS), 2024, pp. 1-6.
- [10] D. Sharma and P. Gupta, "Deep Learning Approaches for Email Security: LSTM and CNN Architectures," IEEE Trans. on Information Forensics and Security, vol. 19, pp. 3456-3468, 2025.